

ros2基本概念

本篇面向已经写过一个 MuJoCo 仿真程序的队员：先讲清楚"为什么需要 ros2"，再动手用 Python 和 C++ 各建立一个最小节点，然后把 ros2 的三种通信方式和 launch 文件过一遍。看完之后，你应该能自己建工作空间、写节点、用话题收发消息，并能看懂别人的 ros2 程序。

一、ros2介绍，以及工作空间结构

1.1 从一个仿真程序说起：控制器层与仿真运行层

我们之前做了仿真程序。从逻辑上抽象分层，可以分成两部分：**控制器层**和**仿真运行层**。

- **控制器层**：负责让机器人阻尼、站立、行走这类命令的计算。它需要知道机器人当前的状态，然后算出应该给机器人什么命令。
- **仿真运行层**：接收控制器计算好的命令并执行，并且向控制器反馈当前仿真中机器人的状态。

两者的关系可以用下面这张简图表示：

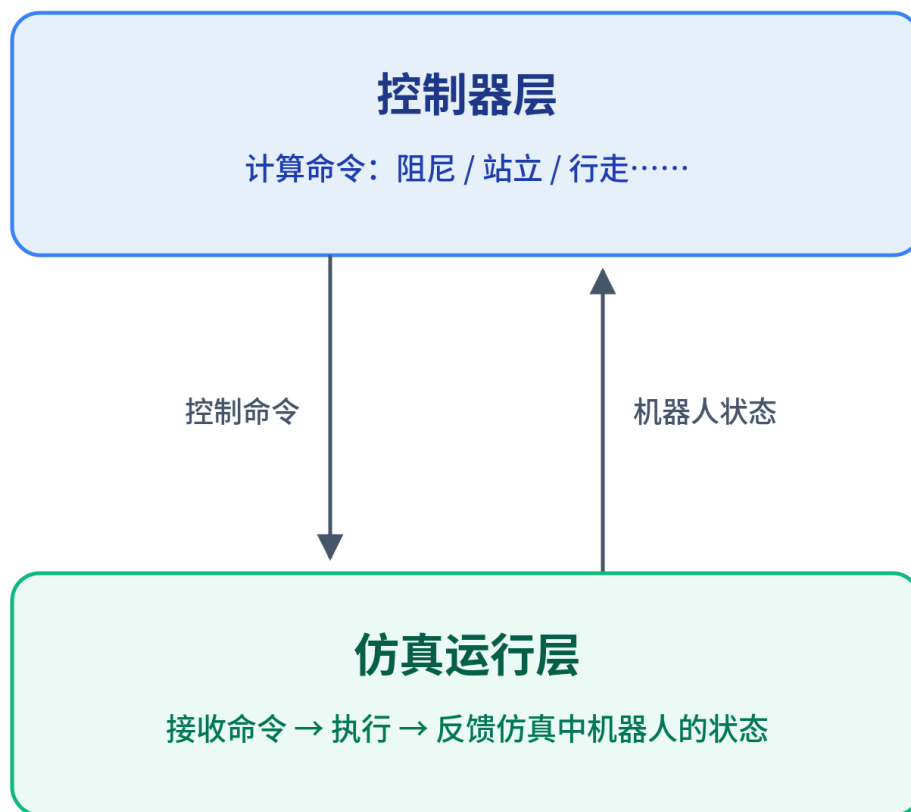


图 1：控制器层与仿真运行层之间的信息交换

从图里可以看到，两层之间交换的信息其实只有两类：控制器层发下去的**控制命令**，和仿真运行层反馈回来的**机器人状态**。

1.2 把仿真换成实机：接口上的启示

接下来我们从接口的角度看这件事。仿真运行层对外做的事情是"接收命令、反馈状态"，而实机做的事情同样是"接收命令、反馈自身状态"。也就是说，从接口来看，**仿真运行层做的就是代替实机**。

因此，我们可以直接把仿真运行层替换成实体机器人的运行程序，控制程序可以不用变；只需要把仿真运行程序切换成实机运行程序，就能用相同的控制器控制实机：

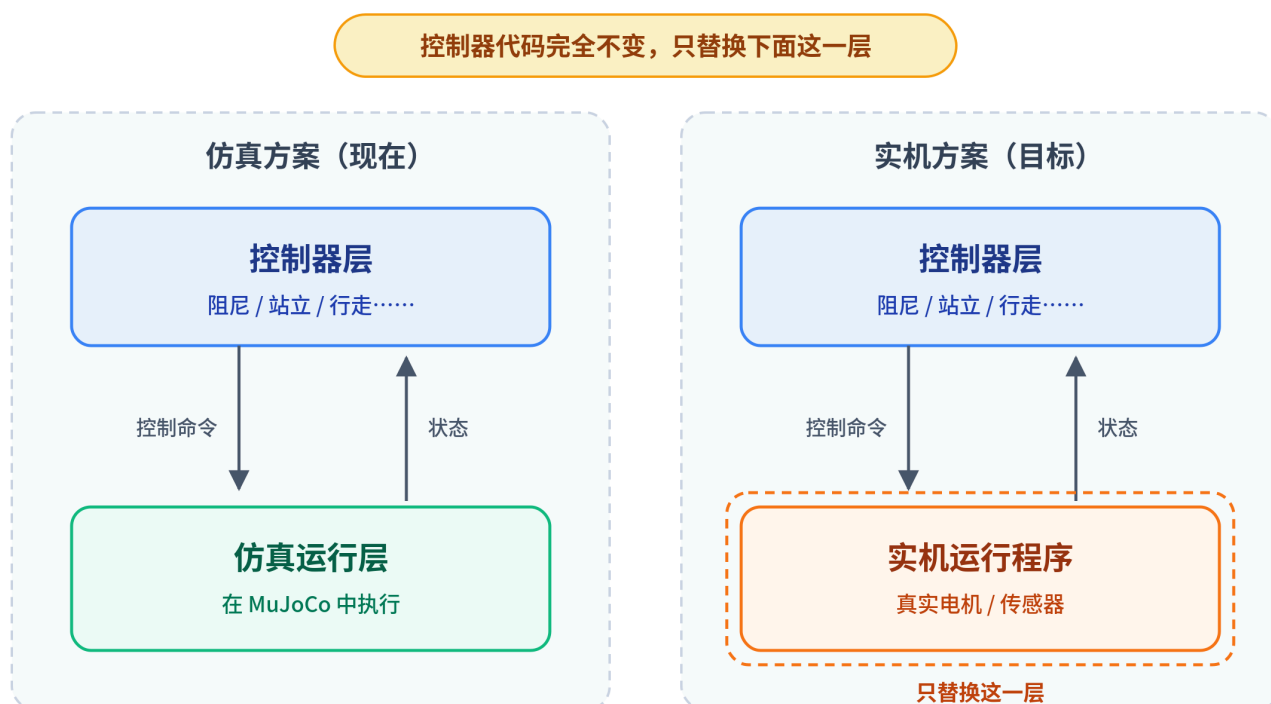


图 2：控制器不变，只把仿真运行层替换成实机运行程序

这在逻辑上是没有任何疑问的。

1.3 问题：分层在代码里体现得不直接

但问题是，大家的程序中，这样的分层体现得不是很直接：**仿真程序和控制程序混合在一个 .py 文件里面**。仿真循环、`mj_step`、控制器计算、状态读取全都缠在一起。

我们怎么才能在代码上把上述的分层、切换这种操作给体现出来呢？于是我们引入 `ros2`。

1.4 `ros2`的作用：把两层拆成可以独立运行的程序

有了 `ros2` 之后，我们可以直接把仿真运行程序和控制程序写成两个能独立运行的程序，两个独立运行的程序之间使用 `ros2` 通信——通信就是交换信息。这种独立运行的程序，我们一般叫做**节点 (node)**。

为了达到上述的效果，我们需要三个节点：**仿真节点、控制器节点、实机节点**。启动控制器节点让它独立运行，这时如果启动仿真节点，控制器就会控制仿真；如果启动实机节点，控制器就会控制实机：

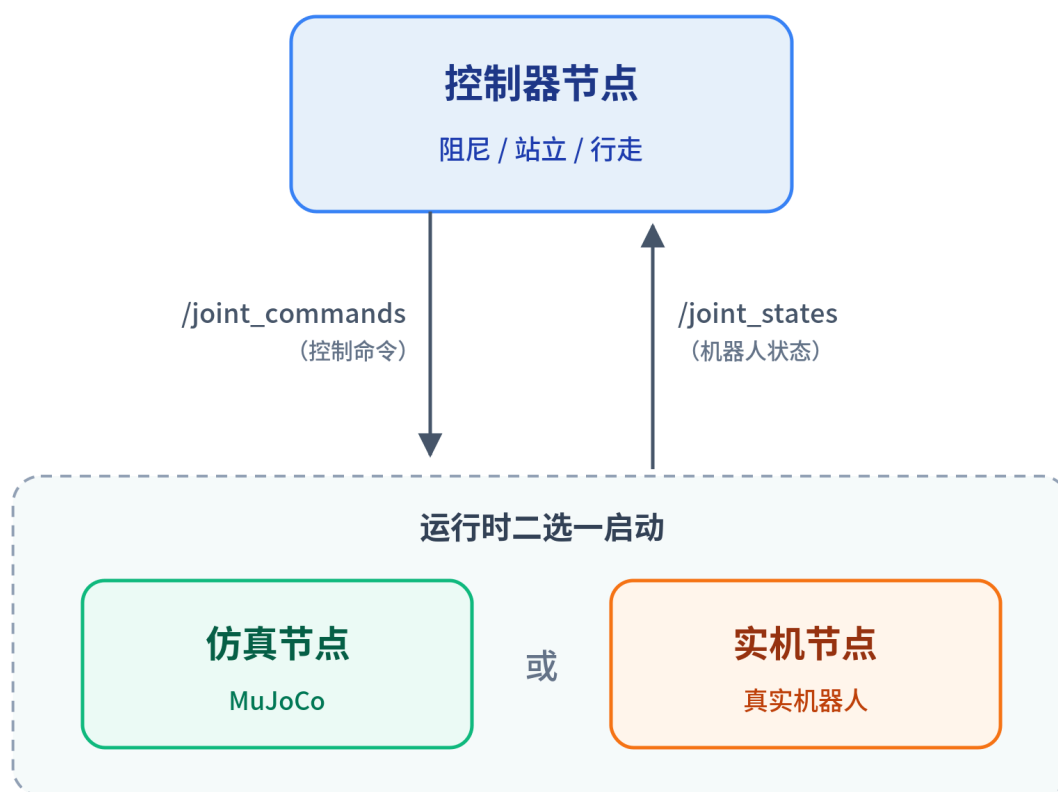


图 3：一个控制器节点，同一时刻只连接仿真节点或实机节点中的一个

这样做了之后，你的仿真程序和控制器程序不再耦合在一块了，因此我们把这叫做**解耦**。

解耦好处的本质是让程序得以复用。比如上面的例子里，我们就让控制器程序得以复用；此外，我们不难联想到，这种得以复用的好处对于代码开源来说也是很好的事情：其他人的程序只要是一个 ros2 节点，我们把它独立运行起来，然后用自己的程序去和开源程序做信息的交换就可以了。

比如说雷达的定位程序：网上开源的 ros2 程序我们可以拿来直接运行，在我们自己的程序里面只要写好接收语句，就能直接利用网上开源的雷达定位程序。

了解了 ros2 的好处和存在的必要，我们就正式开始介绍 ros2。由简入繁，我们先来动手建立一个最小的 ros2 节点，并借此了解 ros2 的骨架结构。

一个提醒：仿真节点和实机节点不要同时启动。它们会订阅同样的指令话题、发布同样的状态话题，控制器就分不清自己在控制谁了。确实需要同时运行时，可以用话题重映射（remap）把名字改开，这是后话，这里先不展开。

1.5 动手：建立第一个 ros2 节点，打印 hello ros2

这一节的目标：从零建好工作空间和功能包，分别用 Python 和 C++ 写一个节点，让它每秒在终端打印一次 `hello ros2`。

1.5.1 先分清三个概念：工作空间、包、节点

在动手之前，先把三个最容易混淆的概念分清楚，后面的命令都是围绕它们展开的：

概念	是什么	例子
工作空间 workspace	一个用来放代码和编译产物的总目录，里面可以装任意多个包	<code>~/ros2_ws</code> ，里面有 <code>src/build/install/log</code>
包 package (功能包)	ros2 里代码组织和编译的最小单位，用包名标识；包里可以放 Python/C++ 代码、 launch、配置等	<code>py_hello</code> 、 <code>cpp_hello</code>
节点 node	包里的一个程序运行起来之后的实例；ros2 通信的主体是节点，运行时用节点名标识	<code>ros2 run cpp_hello talker</code> 启动出来的就是 <code>talker</code> 节点

三者的关系是：工作空间 $\supset$ 包 $\supset$ 节点。展开来说：

- 一个工作空间可以有多个包，比如我们马上就要建一个 Python 包 `py_hello` 和一个 C++ 包 `cpp_hello`；
- 一个包可以定义多个节点，比如 `cpp_hello` 里会有 `hello_node`、`talker`、`listener` 三个可执行程序，每个运行起来都是一个节点；
- 包和源码是"文件"层面的概念，节点是"运行进程"层面的概念：源码写在包里，用 `ros2 run`（或 `launch`）把包里的某个程序跑起来，才产生节点。

1.5.2 创建工作空间和 Python 包

ros2 的代码统一放在一个**工作空间（workspace）**里管理。工作空间本质上就是一个目录，标准结构如下：

```
ros2_ws/
├── src/      # 源码，我们写的功能包都放在这里
├── build/    # 编译中间文件，colcon 自动生成
├── install/  # 编译产物，运行前 source 的就是这里
└── log/     # 编译日志，colcon 自动生成
```

只有 `src` 是我们手动管理的，其余三个目录都是编译时自动生成的。先创建工作空间和第一个功能包：

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_python py_hello --dependencies rclpy
```

解释一下这条命令：

- `ros2 pkg create`：创建一个新的功能包；
- `--build-type ament_python`：这是一个 Python 包（先建它做演示，C++ 包 1.5.6 再建）；
- `py_hello`：包名，后面 `ros2 run` 的时候要用到；
- `--dependencies rclpy`：声明依赖 `rclpy`，它是 ros2 的 Python 客户端库，我们写 Python 节点全靠它。

创建完成后，`src` 下面会多出一个 `py_hello` 目录，结构如下：

```

py_hello/
├── package.xml          # 包的名字、版本、依赖等信息
├── setup.py             # 打包脚本：节点入口、launch 文件在这里注册
├── setup.cfg            # 固定 Python 包的安装路径，基本不用改
├── resource/py_hello   # 一个空标记文件，ament 靠它找到这个包
├── py_hello/           # Python 模块目录，节点代码写在这里
│   └── __init__.py
└── test/               # 自动生成的测试文件，本教程不用管

```

1.5.3 编写 Python 节点代码

在 `py_hello/py_hello/` 目录下新建一个文件 `hello_node.py`（注意是里面那个 `py_hello` 模块目录）：

```

import rclpy
from rclpy.executors import ExternalShutdownException
from rclpy.node import Node

class HelloNode(Node):
    def __init__(self):
        super().__init__('hello_node') # 节点名，ros2 node list 里会显示这个名字
        # 创建一个定时器：每 1 秒调用一次 timer_callback
        self.timer = self.create_timer(1.0, self.timer_callback)

    def timer_callback(self):
        self.get_logger().info('hello ros2')

def main(args=None):
    rclpy.init(args=args)          # 1. 初始化 rclpy
    node = HelloNode()             # 2. 创建节点
    try:
        rclpy.spin(node)           # 3. 让节点一直运行，等待并处理事件
    except (KeyboardInterrupt, ExternalShutdownException):
        pass                        # Ctrl+C 或进程被要求退出，属于正常结束
    finally:
        node.destroy_node()        # 4. 销毁节点
        if rclpy.ok():
            rclpy.shutdown()       # 5. 关闭 rclpy

if __name__ == '__main__':
    main()

```

逐段看这段代码：

1. `class HelloNode(Node)`：定义一个节点类，继承 `ros2` 提供的 `Node` 基类。
`super().__init__('hello_node')` 里的字符串是节点名。

2. `self.create_timer(1.0, self.timer_callback)`: 创建一个定时器, 每 1 秒调用一次 `timer_callback`。节点的能力都是这样"挂"在节点对象上的: 定时器、发布者、订阅者都一样。
3. `self.get_logger().info(...)`: 打印日志。它比 `print` 多了时间戳和节点名, 还能在 `launch` 里统一管理, 是 `ros2` 程序里推荐的打印方式。
4. `rclpy.init / rclpy.spin / rclpy.shutdown`: 这三步是所有 Python 节点的标准生命周期。`spin` 直译是"旋转", 作用是让程序停在这里循环, 一旦有事件 (定时器到点、收到消息.....) 就去调用对应的回调函数; 没有这一句, 程序建立完节点就直接退出了。

1.5.4 注册 Python 节点: 修改 `setup.py`

光写了 Python 文件还不够, 我们得告诉 `ros2` "这个文件里有一个可以运行的节点"。打开包根目录下的 `setup.py`, 找到 `entry_points` 里的 `console_scripts`, 注册节点:

```
entry_points={
    'console_scripts': [
        'hello_node = py_hello.hello_node:main',
    ],
}
```

这一行的格式是 可执行文件名 = 包名.模块名:函数名:

- 等号左边的 `hello_node`, 是将来 `ros2 run` 时用的可执行文件名;
- 等号右边的 `py_hello.hello_node:main`, 表示调用 `py_hello` 模块里 `hello_node.py` 文件的 `main()` 函数。

1.5.5 编译与运行 (Python 版)

回到工作空间根目录编译:

```
cd ~/ros2_ws
colcon build --symlink-install
```

- `colcon build`: 编译整个工作空间;
- `--symlink-install`: 安装时用软链接指向源码。对 Python 节点来说, 改完 `.py` 里的代码直接重新运行就生效, 不用重新编译; 但改 `setup.py` (比如新增节点入口) 必须重新 `colcon build`。

编译完成后, 让当前终端"认识"这个工作空间, 然后运行节点:

```
source install/setup.bash
ros2 run py_hello hello_node
```

应该能看到每秒打印一行:

```
[INFO] [1791129451.435303174] [hello_node]: hello ros2
[INFO] [1791129452.427324844] [hello_node]: hello ros2
[INFO] [1791129453.427314881] [hello_node]: hello ros2
...
```

按 **Ctrl+C** 停止。恭喜，第一个 ros2 节点跑起来了。

重要习惯：每开一个新终端，都要先 `source` 环境，否则 `ros2` 命令和你的包都找不到：

```
source /opt/ros/humble/setup.bash      # 先 source ros2 本体（你装的是别的版本就改版本名）
source ~/ros2_ws/install/setup.bash    # 再 source 自己的工作空间
```

嫌麻烦可以把这两行写进 `~/.bashrc`。

1.5.6 C++ 版：用同样的流程再走一遍

Python 版跑通之后，我们再用 C++ 走一遍完全相同的流程，对比着理解 ros2 的骨架。

再创建一个 C++ 包：

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_cmake cpp_hello --dependencies rclcpp std_msgs
```

和 Python 包的区别主要在前面两个参数：

- `--build-type ament_cmake`：这是一个 C++ 包，用 CMake 编译；
- `--dependencies rclcpp std_msgs`：依赖 C++ 客户端库 `rclcpp`（2.1 节的话题例子还要用到 `std_msgs`，所以一起声明好）。

生成的目录结构：

```
cpp_hello/
├── CMakeLists.txt      # C++ 包的编译说明，地位相当于 Python 包的 setup.py
├── package.xml         # 包信息 + 依赖
├── include/cpp_hello/  # 头文件目录（这个例子用不到）
└── src/               # C++ 源代码目录
```

在 `cpp_hello/src/` 下新建 `hello_node.cpp`：

```
#include <chrono>
#include <functional>
#include <memory>
```

```

#include "rclcpp/rclcpp.hpp"

using namespace std::chrono_literals;

class HelloNode : public rclcpp::Node
{
public:
    HelloNode() : Node("hello_node")
    {
        // 创建一个定时器：每 1 秒调用一次 timer_callback
        timer_ = this->create_wall_timer(
            1s, std::bind(&HelloNode::timer_callback, this));
    }

private:
    void timer_callback()
    {
        RCLCPP_INFO(this->get_logger(), "hello ros2");
    }

    rclcpp::TimerBase::SharedPtr timer_;
};

int main(int argc, char * argv[])
{
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<HelloNode>());
    rclcpp::shutdown();
    return 0;
}

```

对照 Python 版看：

1. `class HelloNode : public rclcpp::Node`：C++ 的节点基类是 `rclcpp::Node`；`Node("hello_node")` 同样是在指定节点名；
2. `create_wall_timer(1s, ...)`：创建一个 1 秒周期的定时器，作用和 Python 的 `create_timer` 一样，`1s` 是 `<chrono>` 提供的时间字面量；
3. `RCLCPP_INFO(this->get_logger(), ...)`：C++ 的日志宏，地位相当于 Python 的 `self.get_logger().info(...)`；
4. `main` 函数：和 Python 的骨架完全对应——`rclcpp::init` 初始化，`std::make_shared<HelloNode>()` 创建节点，`rclcpp::spin` 循环处理事件，`rclcpp::shutdown` 收尾。

然后打开 `CMakeLists.txt`，在最后一行 `ament_package()` 之前加入：

```

add_executable(hello_node src/hello_node.cpp)
ament_target_dependencies(hello_node rclcpp)

install(TARGETS

```

```
hello_node
DESTINATION lib/${PROJECT_NAME})
```

这三段对应 Python 包里的三件事：

- `add_executable`：把 `src/hello_node.cpp` 编译成一个可执行程序（相当于 `setup.py` 里的 `console_scripts`）；
- `ament_target_dependencies`：声明这个程序要链接哪些依赖，比如 `rclcpp`；
- `install`：把可执行程序安装到 `install/cpp_hello/lib/cpp_hello`，`ros2 run` 靠它找到程序。

接着编译运行：

```
cd ~/ros2_ws
colcon build --symlink-install
source install/setup.bash
ros2 run cpp_hello hello_node
```

输出和 Python 版一样：

```
[INFO] [1791132028.889182587] [hello_node]: hello ros2
[INFO] [1791132029.889205704] [hello_node]: hello ros2
[INFO] [1791132030.889251457] [hello_node]: hello ros2
...
```

按 `Ctrl+C` 停止。两种语言、两套工具链，跑出来的是同一种节点，`ros2 node list` 里看到的也都是 `/hello_node`。

Python 包和 C++ 包的对照：

	Python 包 <code>py_hello</code>	C++ 包 <code>cpp_hello</code>
创建命令	<code>--build-type ament_python</code>	<code>--build-type ament_cmake</code>
客户端库	<code>rclpy</code>	<code>rclcpp</code>
源码位置	<code>py_hello/xxx.py</code>	<code>src/xxx.cpp</code>
注册可执行文件	<code>setup.py</code> 的 <code>console_scripts</code>	<code>CMakeLists.txt</code> 的 <code>add_executable + install</code>
运行命令	<code>ros2 run py_hello hello_node</code>	<code>ros2 run cpp_hello hello_node</code>

1.5.7 节点的骨架小结

把两个版本的代码抽象一下，不管用 Python 还是 C++，一个节点的骨架都是同样的五步：

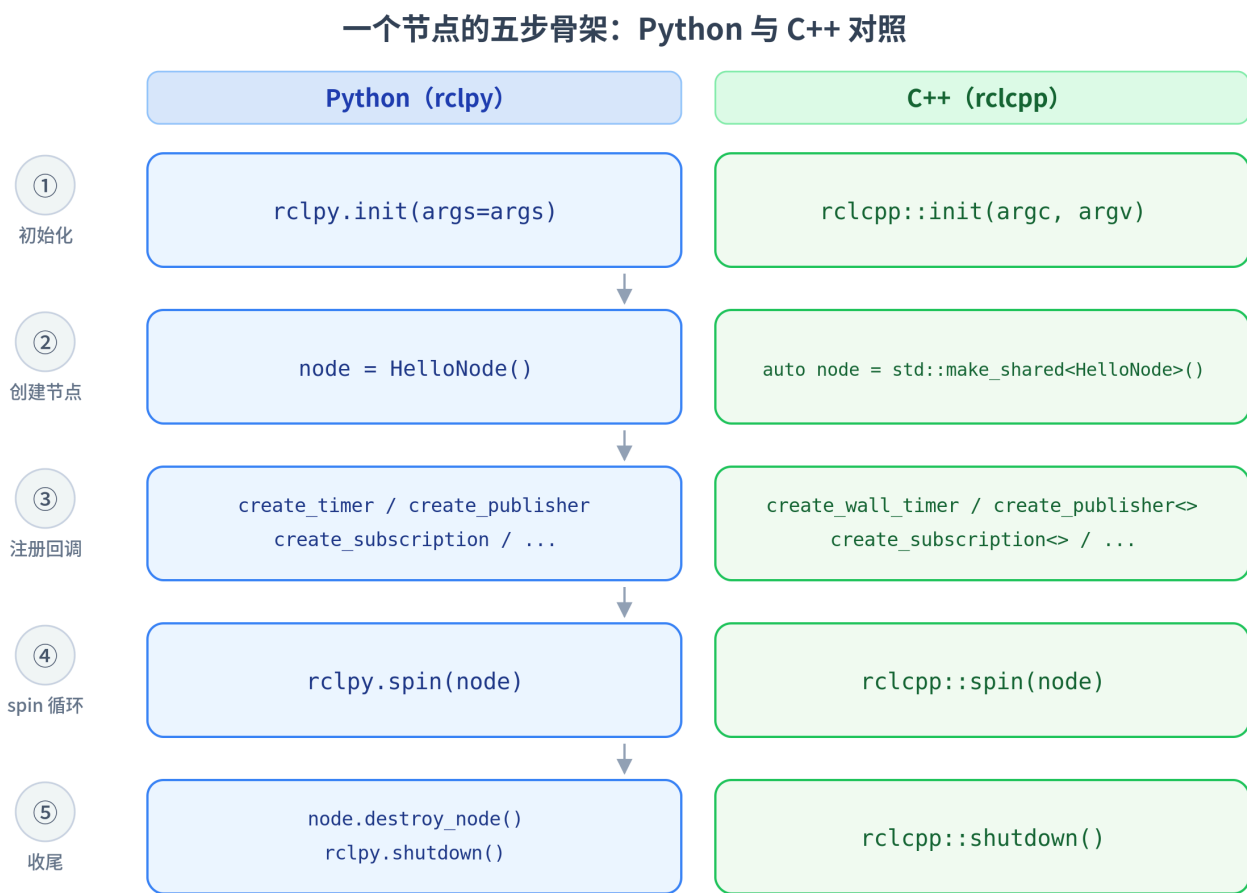


图 4：Python 节点与 C++ 节点的骨架对照

后面写的所有节点，不管多复杂，都是在这五步的框架里往第 3 步加东西。

1.5.8 为什么控制器节点要用 C++ 写

两个 `hello ros2` 节点功能完全一样，那后面的程序到底用哪种语言写？我们的答案是：**控制器节点用 C++ 写。**

原因很直接：针对实时机器人控制的场景，运行效率很重要，C++ 的运行效率相比 Python 更高。具体来说：

- 控制回路通常要跑到几百 Hz 甚至 1 kHz，每个周期里要做矩阵运算、状态估计、关节命令计算，还要保证周期尽量稳定；
- Python 是解释执行，还有 GIL（全局解释器锁）和垃圾回收带来的抖动，控制周期容易忽长忽短；
- C++ 是编译型语言，没有解释器和 GIL 的开销，内存和线程也更容易精确控制，更适合对实时性有要求的场合。

所以从第二节开始，我们的例子程序全部用 C++ 写，正式的控制器节点也用 C++ 实现。Python 也不是没有用武之地：写脚本、做数据分析、快速验证算法都很方便，只是关键的控制回路交给 C++ 更稳妥。

1.6 常用 ros2 命令速查

调试时最常用的几条命令（假设你已经 source 过环境）：

命令	作用
<code>ros2 run <包名> <可执行文件名></code>	运行一个节点，例如 <code>ros2 run cpp_hello hello_node</code>
<code>ros2 node list</code>	列出当前正在运行的节点
<code>ros2 node info /节点名</code>	查看某节点的发布者、订阅者、服务
<code>ros2 topic list</code>	列出当前所有话题；加 <code>-t</code> 同时显示消息类型
<code>ros2 topic echo /话题名</code>	把话题上流过的消息打印到终端，调试神器
<code>ros2 topic info /话题名</code>	查看话题的消息类型、发布者/订阅者数量
<code>ros2 topic hz /话题名</code>	统计话题的发布频率
<code>ros2 topic pub --once /hello std_msgs/msg/String "{data: 'hello'}"</code>	手动发一条消息，测试订阅者
<code>ros2 interface show <消息类型></code>	查看消息类型内部的字段，例如 <code>std_msgs/msg/String</code>
<code>ros2 pkg list / ros2 pkg executables <包名></code>	查看系统里的包 / 某个包里的可执行程序
<code>colcon build --packages-select <包名></code>	只编译某个包（工作空间根目录下执行）
<code>ros2 launch <包名> <launch文件名></code>	用 launch 文件启动（见第三节）

比如运行 `hello_node` 之后，在另一个终端执行：

```
ros2 node list          # 应该看到 /hello_node
ros2 node info /hello_node
```

现在你已经有了骨架，接下来要解决的问题是：两个节点之间怎么交换信息。

二、ros2的通信方式：话题，服务，参数

ros2 节点之间交换信息的方式主要有三种：话题、服务、参数（严格说参数不是"通信"，而是节点对外的配置接口，习惯上放在一起讲）。

2.1 话题通信：单向的信息传递

2.1.1 概念

首先，话题是最简单的 ros2 通信方式。我们写了两个节点——控制器节点和仿真节点，两者需要交换信息。话题是单向的信息传递：首先，控制器程序的控制指令需要发送给仿真运行节点，我们的做法就是：

- 控制器节点创建一个**发布者（publisher）**，然后这个发布者会以一个独特的话题名称向外发布自己的消息；
- 仿真节点创建一个**订阅者（subscriber）**，这个订阅者能够接收一个特定名称的话题消息。

这样，如果发布者发布到话题 `/joint_commands`，订阅者也订阅 `/joint_commands`，即可完成信息从控制器到仿真的传递。反过来，仿真要把机器人状态反馈给控制器，只需要把角色对调：仿真节点发布 `/joint_states`，控制器节点订阅 `/joint_states`。所以"控制器和仿真交换信息"这件事，在 `ros2` 里就是两条方向相反的话题：

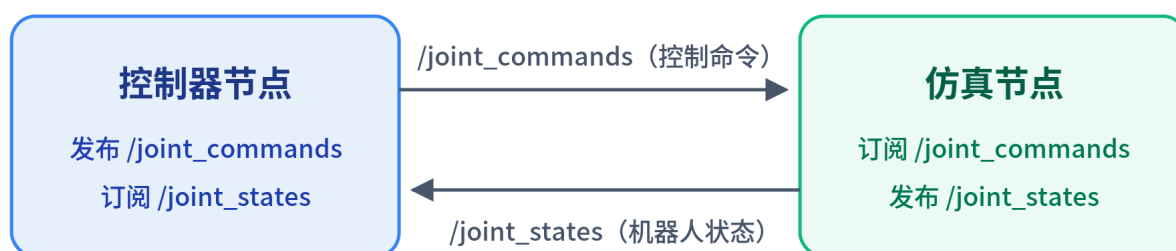


图 5：控制器节点与仿真节点之间的两条话题

这里有两个要点：

1. 话题的名字必须完全一致（区分大小写和斜杠），消息类型也必须一致；
2. 发布者只管发，不关心有没有人在听；订阅者只管收，不关心是谁发的。两边没有互相"调用"的关系，这也是解耦在通信上的体现。

回顾 1.4 的图：控制器节点启动之后，它并不需要知道对面是仿真节点还是实机节点；只要对面订阅同样的 `/joint_commands`、发布同样的 `/joint_states`，控制器就能工作。这就是"同一套控制器，仿真和实机都能用"在通信层面的实现。

2.1.2 例程：talker 与 listener (C++ 版)

下面用一个更简单的例子理解话题的代码书写：节点 `talker` 发布消息 `"hello"`；另一个节点 `listener` 接收 `talker` 的消息，然后输出到终端。按照 1.5.8 的约定，从这一节开始例子都用 C++ 写。

我们在 1.5.6 建立的 C++ 包 `cpp_hello` 里把这两个节点加上。在 `cpp_hello/src/` 下新建 `talker.cpp`：

```

#include <algorithm>
#include <chrono>
#include <functional>
#include <memory>
#include <string>

#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"
  
```

```

class Talker : public rclcpp::Node
{
public:
    Talker() : Node("talker")
    {
        // 声明参数：发布的内容和周期，默认每秒发一次 "hello" (见 2.3)
        this->declare_parameter<std::string>("message", "hello");
        this->declare_parameter<int>("period_ms", 1000);
        message_ = this->get_parameter("message").as_string();
        const int period_ms =
            std::max(static_cast<int>(this->get_parameter("period_ms").as_int()),
1);

        // 创建发布者：消息类型 String，话题名 hello，队列长度 10
        publisher_ = this->create_publisher<std_msgs::msg::String>("hello",
10);
        timer_ = this->create_wall_timer(
            std::chrono::milliseconds(period_ms),
            std::bind(&Talker::timer_callback, this));
    }

private:
    void timer_callback()
    {
        auto msg = std_msgs::msg::String();
        msg.data = message_;
        publisher_->publish(msg);
        ++count_;
        RCLCPP_INFO(this->get_logger(), "发布第 %zu 条: %s", count_,
msg.data.c_str());
    }

    rclcpp::Publisher<std_msgs::msg::String>::SharedPtr publisher_;
    rclcpp::TimerBase::SharedPtr timer_;
    std::string message_;
    size_t count_ = 0;
};

int main(int argc, char * argv[])
{
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<Talker>());
    rclcpp::shutdown();
    return 0;
}

```

再新建 `listener.cpp`:

```

#include <functional>
#include <memory>

```

```

#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"

class Listener : public rclcpp::Node
{
public:
    Listener() : Node("listener")
    {
        // 创建订阅者：消息类型 String，话题名 hello，收到消息调用 callback
        subscription_ = this->create_subscription<std_msgs::msg::String>(
            "hello", 10,
            std::bind(&Listener::callback, this, std::placeholders::_1));
    }

private:
    void callback(const std_msgs::msg::String::SharedPtr msg) const
    {
        RCLCPP_INFO(this->get_logger(), "收到: %s", msg->data.c_str());
    }

    rclcpp::Subscription<std_msgs::msg::String>::SharedPtr subscription_;
};

int main(int argc, char * argv[])
{
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<Listener>());
    rclcpp::shutdown();
    return 0;
}

```

两个文件各看两处关键地方就够了：

- **发布端：** `create_publisher<std_msgs::msg::String>("hello", 10)` 说明"我要在 `hello` 这个话题上发 `std_msgs::msg::String` 类型的消息，队列长度 10"；然后在定时器回调里用 `publisher_->publish(msg)` 真正发出去。C++ 里消息类型写在尖括号的模板参数里，话题名和队列长度是普通参数。
- **订阅端：** `create_subscription<std_msgs::msg::String>("hello", 10, 回调)` 说明"我订阅 `hello` 话题，收到消息就调用 `callback`"。`std::bind(&Listener::callback, this, std::placeholders::_1)` 只是把成员函数 `callback` 包装成回调函数；回调不是我们手动调用的，`spin` 收到消息后会自动调用它。

`std_msgs::msg::String` 来自 `ros2` 自带的 `std_msgs` 包，它只有一个字段 `data`，就是我们要传的字符串。

接着把这两个可执行文件注册到 `CMakeLists.txt`（在 1.5.6 已经加过 `hello_node` 的基础上，追加两段并更新 `install`）：

```

add_executable(talker src/talker.cpp)
ament_target_dependencies(talker rclcpp std_msgs)

```

```
add_executable(listener src/listener.cpp)
ament_target_dependencies(listener rclcpp std_msgs)

install(TARGETS
  hello_node
  talker
  listener
  DESTINATION lib/${PROJECT_NAME})
```

`install(TARGETS ...)` 里要带上所有可执行文件，漏掉的程序 `ros2 run` 是找不到的。

重新编译，然后 `source`：

```
cd ~/ros2_ws
colcon build --symlink-install
source install/setup.bash
```

打开两个终端（都先 `source` 好环境），一个运行 `talker`，一个运行 `listener`：

```
# 终端 A
ros2 run cpp_hello talker
```

```
# 终端 B
ros2 run cpp_hello listener
```

终端 A 每秒打印一条"发布"，终端 B 每秒打印一条"收到"：

```
# 终端 A
[INFO] [...] [talker]: 发布第 1 条: hello
[INFO] [...] [talker]: 发布第 2 条: hello
...

# 终端 B
[INFO] [...] [listener]: 收到: hello
[INFO] [...] [listener]: 收到: hello
...
```

先启动哪个都行。`ros2` 会自动完成节点之间的"发现" (discovery)，不需要谁先等谁。

再开第三个终端，可以观察和管理这个话题：

```
ros2 topic list -t      # 能看到 /hello [std_msgs/msg/String]
ros2 topic info /hello  # 发布者 1 个、订阅者 1 个
ros2 topic echo /hello  # 把 /hello 上流过的消息实时打印出来
ros2 topic hz /hello    # 统计发布频率，应该接近 1 Hz
```

2.1.3 话题通信的两个角色

角色	创建方式	关键动作
发布者 Publisher	<code>create_publisher<消息类型>("话题名", 队列长度)</code>	调用 <code>publish(msg)</code>
订阅者 Subscriber	<code>create_subscription<消息类型>("话题名", 队列长度, 回调)</code>	收到消息后自动调用回调函数

队列长度写 10，表示"最多缓存 10 条消息"：如果发布太快、订阅者处理不过来，新的消息会挤掉旧的消息。现在统一写 10 就行，更复杂的场景（比如传感器高频数据）会用到 QoS 配置，需要时再查。

2.2 服务通信：双向的请求与回应

2.2.1 概念

区别于话题，服务是双向的。在服务通信中，通信双方其中一个我们称为**客户端（Client）**，另一个称为**服务端（Server）**。两者的作用非常相称其名称：客户端向服务端提出一个要求，服务端完成要求之后将结果发回客户端。因此消息的传递方式是：客户端节点发布消息，服务端接收消息后，向客户端返回一个消息。



图 6：服务通信中的请求与响应

一个服务由三样东西定义：服务名称、请求（Request）的数据结构、响应（Response）的数据结构。客户端发请求之后会等一个响应；服务端处理完请求后，必须回一个响应，一次服务调用才算结束。

2.2.2 例子：用服务做"握手"

这个机制可以用来做"握手"。比如说我们现在有两个节点：导航节点和遥控器节点。遥控器节点想让导航节点开始导航，如果用话题传递这个消息的话，如果出现这个消息丢失，是不是非常尴尬。

那么我们如果用服务类型的通信，就可以要求导航节点说一声"收到了"：遥控器节点向导航节点发送消息，如果导航节点收到了，就回信一个"收到了"，遥控器节点收到这个回信之后，就知道导航节点收到消息了，就可以进行下一步判断。

ros2 的通信底层实际上是网络通信，网络当然可以走无线，因此实际上在同一个局域网内，即使是不同机器上的 ros2 节点，也是可以相互发消息的。在同一个机器上两个节点或许很少丢失消息，但是如果是跨机器，这种"握手"却是必要的。

跨机器的前提：两台机器在同一个局域网、能互相 ping 通，并且 ROS_DOMAIN_ID 一致（默认都是 0）。不满足这些条件时，节点互相"发现"不到。

2.2.3 话题和服务怎么选

	话题 Topic	服务 Service
方向	单向：发布者 → 订阅者	双向：客户端 ⇌ 服务端
有没有回应	没有	有，服务端必须回一个响应
典型用途	持续不断的数据流：关节指令、机器人状态、雷达数据	一次性的请求：开始导航、切换模式、保存地图
丢失消息的后果	丢一帧还有下一帧，通常能接受	请求丢了就没有响应，客户端能发现

一句话记法：话题像广播，服务像打电话。

2.2.4 服务：会"描述需求"就够了

培训阶段不需要你手写服务的代码，但你要能把需求给 AI 描述清楚。描述一个服务，至少交代这四件事：

- 1. 谁是客户端、谁是服务端；
- 2. 服务名是什么；
- 3. 请求里要带什么数据、响应里要带什么数据；
- 4. 客户端什么时候调用，拿到响应之后怎么处理。

例如可以这样向 AI 提要求：

帮我用 C++（ros2 Humble、rclcpp）写两个节点。服务端是导航节点，提供一个名为 /start_navigation 的服务：请求里包含目标点 x、y，响应里包含 success（是否成功）和 message（文字说明）；收到请求后打印目标点，并返回 success=true。客户端是遥控器节点，启动后每隔 3 秒调用一次这个服务，并打印响应内容。

如果只是"通知一下"、不需要额外数据，可以用 ros2 自带的 std_srvs/srv/Trigger 类型：请求为空，响应只有 success 和一个字符串 message，很适合做上面说的"握手"。

遇到别人写好的服务时，可以用命令先看清楚它：

```
ros2 service list -t           # 列出所有服务及类型
ros2 service type /服务名     # 查看某个服务的类型
ros2 service call /服务名 <服务类型> "<请求内容>" # 手动调一次，调试用
```

2.3 参数：可以在线调整的变量

有时候有些参数我们需要在线调整，或者每次启动时尝试不同的参数，ros2 允许将某些变量设置为**参数 (parameter)**。

参数不是节点之间的通信，而是**节点对外的配置接口**：每个节点可以声明若干个参数，外部（命令行、launch 文件、其他程序）可以读取和修改它们。

```
this->declare_parameter<std::string>("message", "hello"); // 声明参数，默认值 "hello"
this->declare_parameter<int>("period_ms", 1000);          // 默认 1000 ms

message_ = this->get_parameter("message").as_string();    // 读取参数
const int period_ms = this->get_parameter("period_ms").as_int();
```

`talker.cpp` 开头这几行就是把“发什么内容、多久发一次”做成了参数。

启动节点时可以在命令行指定参数：

```
ros2 run cpp_hello talker --ros-args -p message:=你好ros2 -p period_ms:=500
```

运行过程中也可以查看和修改：

```
ros2 param list           # 列出各节点声明了哪些参数
ros2 param get /talker message # 读取某个参数的值
ros2 param set /talker message 你好
```

注意：我们的 `talker` 只在启动时读一次参数，所以 `param set` 虽然会显示成功，但不会立刻改变发布内容。想让参数在线生效，需要在代码里每次使用前重新读取，或者写参数回调——这就是更进阶的内容了。

参数适合放这些量：控制器的 `kp`、`kd`，仿真步长，最大速度，话题名字，调试开关.....原则是：**凡是“希望不改代码就能改”的量，都可以做成参数**。给 AI 提需求时，也可以直接说“把 `kp`、`kd` 做成参数，默认值是.....”，它就知道该怎么写了。

在第三节的 launch 文件里，我们会用 launch 文件统一给节点传参数，比每次手敲一长串 `-p` 方便得多。

三、launch 文件

3.1 要解决的问题

正常我们启动一个 ros2 节点的时候，需要使用命令 `ros2 run <包名> <可执行文件名>`；如果要启动多个节点，就要多开几个终端、多用几次 `ros2 run`。这是不方便的，我们可以写一个 launch 文件，通过这一个文件来启动所有你想启动的节点。

3.2 写一个 launch 文件：一键启动 talker 和 listener

我们以一键启动最开始的 `talker` 和 `listener` 两个节点为例子。

在 `cpp_hello` 包根目录下新建一个 `launch` 目录，里面新建 `talk_listener.launch.py`：

```
cpp_hello/
├── launch/
│   └── talk_listener.launch.py    # 新增
├── src/
│   ├── hello_node.cpp
│   ├── talker.cpp
│   └── listener.cpp
├── CMakeLists.txt
└── package.xml
```

文件内容如下：

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='cpp_hello',
            executable='talker',
            name='talker',
            output='screen',
        ),
        Node(
            package='cpp_hello',
            executable='listener',
            name='listener',
            output='screen',
        ),
    ])
])
```

逐行解释：

- launch 文件有 XML、YAML、Python 三种写法，我们统一用 Python：它最灵活，也是 ros2 社区最常用的写法；
- `generate_launch_description()` 是固定的入口函数，`ros2 launch` 会调用它；

- 一个 `Node(...)` 描述一个要启动的节点：`package` 是包名，`executable` 是 `CMakeLists.txt` 里注册的可执行文件名，`name` 是节点名（可以和可执行文件名不同），`output='screen'` 表示把日志打印到当前终端；
- 两个 `Node` 放进同一个列表，`ros2 launch` 就会把它们一起启动。

3.3 注册 launch 文件并运行

launch 文件也需要在 `CMakeLists.txt` 里注册，否则 `ros2` 在 `install` 目录里找不到它。在最后一行 `ament_package()` 之前追加一段：

```
install(DIRECTORY launch
        DESTINATION share/${PROJECT_NAME})
```

它的意思是把包里的 `launch` 目录安装到 `share/cpp_hello/launch`，`ros2 launch` 就是从这里找 launch 文件的。Python 包是在 `setup.py` 的 `data_files` 里做同样的事情，C++ 包则统一写在 `CMakeLists.txt` 里。

然后重新编译、source、运行：

```
cd ~/ros2_ws
colcon build --symlink-install
source install/setup.bash
ros2 launch cpp_hello talk_listener.launch.py
```

输出大致如下：launch 先报告启动了哪些进程，然后是 talker 和 listener 的日志：

```
[INFO] [launch]: All log files can be found below ...
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [talker-1]: process started with pid [...]
[INFO] [listener-2]: process started with pid [...]
[talker-1] [INFO] [...] [talker]: 发布第 1 条: hello
[listener-2] [INFO] [...] [listener]: 收到: hello
[talker-1] [INFO] [...] [talker]: 发布第 2 条: hello
[listener-2] [INFO] [...] [listener]: 收到: hello
...
```

按 `Ctrl+C` 会把这次 launch 启动的所有节点一起停掉，不用再挨个终端去关。

小提示：--symlink-install 下，修改已有 launch 文件的内容直接生效，不用重新编译；但新增、删除或改名 launch 文件，需要重新 `colcon build`。

3.4 在 launch 文件里写参数

launch 文件不只是用来启动节点，还可以写入参数。比如我们想把 talker 发的内容和频率改掉，只要在对应的 `Node(...)` 里加一个 `parameters`：

```
Node(
    package='cpp_hello',
    executable='talker',
    name='talker',
    output='screen',
    parameters=[{
        'message': '你好, ros2',
        'period_ms': 500,
    }],
),
```

`listener` 的 `Node(...)` 不变。再运行同一个 `launch` 文件，输出就变成每 0.5 秒一条：

```
[talker-1] [INFO] [...] [talker]: 发布第 1 条: 你好, ros2
[listener-2] [INFO] [...] [listener]: 收到: 你好, ros2
[talker-1] [INFO] [...] [talker]: 发布第 2 条: 你好, ros2
[listener-2] [INFO] [...] [listener]: 收到: 你好, ros2
...
```

这里的参数和 2.3 节命令行的 `--ros-args -p message:=... -p period_ms:=...` 完全等价，只是写进了文件里：每次启动自动带上，不用手敲。

参数多了以后，还可以把它们单独写在一个 YAML 文件里，在 `launch` 里这样引用：

```
parameters=['/path/to/params.yaml'] # 指向参数文件（建议写绝对路径）
```

YAML 的具体格式需要时再查即可，这里先掌握上面的内联写法。

3.5 小结

到这里，一个最小的 `ros2` 工程链路就完整了：

- 用工作空间和功能包管理代码，`colcon build` 编译，用 `source` 让终端认识它；
- 用节点把控制器、仿真、实机拆成能独立运行的程序，它们通过话题交换信息；
- 需要“确认对方收到”的场合用服务；希望不改代码就能调整的量做成参数；
- 用 `launch` 文件一键启动所有节点，并统一传入参数。
- 控制回路用 C++ 实现，保证实时性；Python 适合写脚本、做数据分析和快速验证算法。

回到最开始的目标：控制器节点和仿真/实机节点解耦之后，控制器只需要认话题，不需要知道对面是谁。以后要上实机，就是把仿真节点换成实机节点；以后想用别人的开源节点（比如雷达定位），只要它收发约定好的话题，运行起来就能直接配合——这就是 `ros2` 带来的复用。